

# KAIROS

Knowledge-Augmented Insight & Retention Optimization System

*The right insight, at the right moment.*

*Formal Project Title*

**An Explainable AI-Driven Customer Churn Intelligence Framework for Predictive  
and Actionable Retention**

**Final Year Major Project Documentation**

B.E. / B.Tech — Semester VII & Semester VIII

Academic Duration: August 2026 – October 2026 (Sem VII) | January 2027 – March 2027 (Sem VIII)

Document Version 2.0

Prepared: August 2026

## 0. Naming Note

---

This project has two names, used deliberately for two different purposes:

- Formal project title — “An Explainable AI-Driven Customer Churn Intelligence Framework for Predictive and Actionable Retention.” Use this exact title on your SRS, interim report, final report, and any paper submission, since this is the academically-styled title reviewers, guides, and journals/conferences expect.
- Codename — “Kairos.” Use this everywhere else: your GitHub repo name, demo, resume, viva conversation, and throughout this document, for brevity and memorability.

Both names refer to the exact same system. This document uses “Kairos” throughout for readability, and the formal title where a section is explicitly about report/paper submission.

## 1. Executive Summary

---

Kairos is an explainable, conversational customer churn intelligence platform. It predicts which customers are likely to churn, explains why in plain business terms, and lets people ask follow-up questions about that data and those explanations directly, in natural language, rather than requiring them to read a dashboard or write SQL. The platform is built in two tightly connected layers: a governed data and machine-learning core (customer, usage, support, and transaction data; an interpretable churn model with SHAP-based explanations; drift monitoring) and a conversational layer on top of it (a hybrid SQL + semantic retrieval engine that answers natural-language questions, grounded in real data and real model explanations rather than free-form guessing).

The result is a system where a support agent, analyst, or executive can ask, in plain English, “Which customers in the West region are likely to churn next month, and why?” and receive an answer grounded in real SQL execution, semantic retrieval over support tickets, and model-generated explanations — not a hallucinated guess.

The name Kairos (Greek: the right or opportune moment) reflects the project's core value proposition: surfacing the right insight, explained, at the right moment for the right audience — rather than requiring people to build a new dashboard or write SQL every time a new question arises. Kairos is also a backronym: Knowledge-Augmented Insight & Retention Optimization System — knowledge-augmented because every answer is grounded in real retrieved data and model explanations rather than free-form generation, and insight/retention-optimization because the entire platform exists to turn churn predictions into concrete retention action.

This document consolidates the problem statement, literature review direction, system architecture, novelty claims, a two-semester phased execution plan (August–October 2026 for Semester VII, January–March 2027 for Semester VIII), curated learning resources, and a verified reference list, aligned to your institution's expected Sem VII checkpoint (literature review + working prototype, early October 2026)

and Sem VIII final submission (full report + deployed system, end of March 2027). It also outlines a realistic path toward a Scopus-indexed or IEEE publication that can strengthen your job applications.

Positioning note: Kairos is written and sequenced for a classic Data Scientist job search — the core, load-bearing story is explainable churn modelling, rigorous evaluation, and business-insight storytelling through BI dashboards. The conversational hybrid-RAG layer is a genuine differentiator and is kept in the plan, but it is treated throughout this document as Module B — a bonus that shows range — built on top of a solid analytics core, not a replacement for it. Section 3.1 explains why this ordering matters and how to carry it into interviews.

## 2. Design Philosophy — Two Layers, One Platform

---

Kairos is deliberately built as two layers that depend on each other, rather than a single flat pipeline. This split is what makes the project substantial enough for two full semesters of distinct, defensible work, and it maps naturally onto “literature review → design → prototype → full system → evaluation → publication.”

- The governed data and ML core gives Kairos a real, verifiable substrate to reason over: a proper relational schema (customers, subscriptions, usage\_logs, support\_tickets, transactions, churn\_predictions, model\_registry), a production-style ML pipeline with SHAP explainability and drift monitoring, and a genuine business narrative — something meaningful and checkable for the conversational layer to talk about.
- The conversational layer gives Kairos its “right moment” value proposition: a query router, hybrid SQL + vector retrieval engine, and a formal evaluation harness (text-to-SQL execution accuracy, RAGAS-style faithfulness/relevance) that turn “chat with your data” from a UI gimmick into a measurable, defensible engineering contribution, rather than a system that hallucinates plausible-sounding numbers.

A system built with only the first layer is a solid but unremarkable churn-analytics project with static dashboards — someone still has to know which chart to look at. A system built with only the second layer is a capable RAG engine with nothing meaningful at stake if an answer is wrong. Kairos is designed so each layer fixes the other's weakest point, and together they justify two full semesters of work distributed across genuinely different skill areas: data engineering, classical ML + XAI, LLM/RAG engineering, BI/dashboarding, evaluation research, and deployment.

## 3. Combined Problem Statement

---

Organisations running subscription or usage-based businesses (SaaS, telecom, banking, retail, insurance) generate rich customer data across billing, usage, and support systems, and many now have churn-prediction models sitting on top of that data. Yet the people who most need churn insight day-to-day — support agents, retention managers, and executives — are rarely SQL-literate or comfortable navigating

multi-panel BI dashboards. This creates a persistent “last-mile” gap between predictive insight and predictive action.

At the same time, generic “chat with your data” tools (including uploading a CSV to a general-purpose chatbot) are unsuitable for this setting because they:

- Hallucinate numbers and cannot reliably perform aggregation, joins, or precise lookups over large, multi-table schemas;
- Have no notion of a live ML model, so they cannot answer “why” questions grounded in an actual prediction (e.g., which SHAP features drove a specific customer's risk score);
- Give no indication of answer confidence, data freshness, or model drift, so a plausible-sounding answer may be built on a stale or degraded model — a risk explicitly flagged in recent literature on conversational interfaces for governed predictive analytics;
- Are rarely evaluated formally; most demos are judged by a handful of manually inspected examples rather than a reproducible accuracy/faithfulness benchmark.

Kairos addresses this gap by building a conversational, explainable, drift-aware analytics layer directly on top of a governed churn-intelligence data and ML platform, with a formal evaluation methodology borrowed from both the text-to-SQL and RAG evaluation literature.

### 3.1 Positioning for a Classic Data Scientist Role

This matters enough to state explicitly: a classic Data Scientist interview panel weighs statistical rigor, business-insight translation, and dashboard storytelling far more heavily than retrieval-engine or LLM-orchestration engineering. So even though this document keeps the full two-module scope (it's still a strong, differentiated project), the priority order for your time, your report emphasis, and your interview narrative should be:

1. Core (protect this, do it deeply): governed data model, leakage-safe feature engineering, model selection and validation, SHAP-based interpretation, and translating that interpretation into concrete business recommendations — shown through the two BI dashboards.
2. Module B (bonus, build it, but it can flex if time is tight): the hybrid RAG conversational layer, the explanation-faithfulness metric, and the publication. Genuinely valuable for range and for standing out, but not the piece an interviewer will spend the most time on.

Practical effect on how you should talk about Kairos later: open with “I built an explainable churn model, validated it properly, and turned the SHAP output into segment-level retention recommendations delivered through two stakeholder-specific dashboards” — then close with “...and I additionally layered a natural-language query interface on top so non-technical stakeholders could ask questions directly.” That ordering, not the reverse, is what a classic DS interviewer expects to hear.

## 4. Objectives

---

1. Build a governed relational data layer for customer churn intelligence (customers, subscriptions, usage, support, transactions, predictions, model registry) with realistic, leakage-aware synthetic augmentation of a public seed dataset.
2. Train an interpretable churn-prediction pipeline (baseline logistic regression + XGBoost/LightGBM) with SHAP-based global and per-customer explanations, tracked via MLflow, with drift detection (PSI/KS-test) and champion/challenger promotion.
3. Translate SHAP outputs and cohort/segment analysis into concrete, plain-English business recommendations, and deliver them through two audience-specific BI dashboards (Power BI for operations, Tableau for executives) — this is the primary insight-delivery objective of the project.
4. [Module B] Design and implement a hybrid retrieval engine (structured SQL retrieval + semantic vector retrieval over free-text columns via pgvector) with an intelligent query router that classifies incoming natural-language questions as aggregation, lookup, semantic, or hybrid.
5. [Module B] Extend the router so it can retrieve and reason over model-generated artefacts — SHAP driver explanations and drift/model-confidence metadata — not just raw table rows, so conversational answers can explain “why” and flag “how reliable.”
6. Expose the system through a JWT-secured Flask REST API and, as a bonus interface, a conversational layer that sits above the two dashboards for stakeholders who prefer to ask questions in plain English instead of navigating panels.
7. Build a formal, reproducible evaluation framework: model metrics (ROC-AUC, PR-AUC, calibration, expected-revenue-saved) as the primary axis, plus — for Module B — text-to-SQL execution accuracy, RAGAS-style faithfulness/relevance, and a novel explanation-faithfulness metric.
8. Conduct a literature review, a market/job-relevance study, and a small user-need survey to justify the project's real-world relevance, and use these to identify a defensible novelty claim suitable for a Scopus-indexed or IEEE publication.
9. Deliver a fully documented, containerized, portfolio- and interview-ready capstone with a working demo, final report, and a submitted (or in-review) research paper.

## 4.1 What “Insight” Means for Kairos — Viva-Ready Examples

A classic DS project is judged heavily on whether you can go from a model to a decision. Aim to produce (and put directly into your dashboards and report) concrete findings in this style, derived from your own SHAP and cohort analysis once the model is trained:

- Driver insight: “Failed-payment count in the last 90 days is the single strongest SHAP driver of churn for month-to-month customers — stronger than tenure or monthly charges.”
- Cohort insight: “Customers who signed up via [channel/plan] and had 2+ support tickets in their first 60 days churn at roughly Nx the base rate.”

- Segment recommendation: “For the ‘high value, disengaging’ segment, a proactive outreach at day 45 of declining usage is the highest-leverage intervention point, based on the usage-trend feature’s SHAP contribution.”
- Business-impact framing: “Acting on the top-risk decile identified by the model corresponds to an estimated ₹X / \$X in retained monthly revenue, assuming a Y% success rate on outreach.”

These four sentence patterns — driver, cohort, segment recommendation, business impact — are what should appear on your dashboards, in your final report, and in your mouth during the viva when asked “what did you find?”

## 5. Literature Review — Direction, Themes, and Seed Papers

---

Your Sem VII checkpoint requires a completed literature review, so this section gives you a structured starting map with real, recent papers across the three themes Kairos sits between. Section 15 gives the full, verified IEEE Xplore / Scopus-indexed reference list to cite formally; use this section for the narrative synthesis and gap statements. Aim for 25–35 papers total once you expand outward via citation search, and treat the gaps identified in italics below as your novelty anchors.

### 5.1 Theme A — Explainable Churn Prediction

This theme is the most mature: SHAP-based explanation of tree-ensemble churn models (XGBoost, LightGBM, Random Forest, CatBoost) is now a well-established pattern in both academic and industry literature, consistently applied to telecom, banking, e-commerce, and SaaS datasets with strong reported performance (AUC-ROC commonly in the 0.85–0.93 range).

Gap you can cite: this entire body of work treats explanations as a static, human-read visualization (a SHAP bar chart in a notebook or dashboard). None of it treats an explanation as a retrievable, queryable object that a downstream conversational system can reason over — that is Kairos's opening.

### 5.2 Theme B — Natural-Language-to-SQL and Conversational Database Interfaces

This theme spans two decades of work, from early rule-based NL-to-SQL parsers through modern LLM-based text-to-SQL systems and conversational (multi-turn) extensions. Execution accuracy and schema-linking robustness remain the central open problems.

Gap you can cite: this body of work focuses on translating a question into a correct query against a static database. It does not address what happens when the “database” also contains live, versioned model predictions, confidence scores, and drift state — Kairos's second opening.

### 5.3 Theme C — Retrieval-Augmented Generation over Structured/Enterprise Data

This theme is the newest and fastest-moving: RAG systems originally designed for unstructured documents are being adapted for structured, multi-entity, and enterprise tabular data, with hybrid architectures combining dense retrieval, SQL execution, and knowledge-graph-style structuring.

Gap you can cite: current structured-RAG systems retrieve over enterprise tables and documents, but none are designed to retrieve over a live ML-serving layer where the retrievable “data” includes model predictions, explanations, and drift/confidence metadata — Kairos's third opening, and the direct synthesis point across all three themes.

## 5.4 How to use this section

- Write a short synthesis paragraph per theme (not a list) explaining what is established, what techniques are mature enough to reuse directly (SHAP, XGBoost, hybrid SQL + vector retrieval, RAGAS-style metrics), and what is missing.
- End the literature review chapter with an explicit “research gap” paragraph that states the three gaps above in your own words — this becomes the opening paragraph of your eventual paper's Introduction.
- Expand the seed list in Section 15 using Scopus, IEEE Xplore, and Google Scholar backward/forward citation search from each paper; target roughly 60% papers from 2024–2026 to demonstrate currency.

## 6. Novelty and Research Contribution

---

A plain churn-prediction dashboard or a generic RAG-over-spreadsheet tool would each be, on their own, a well-covered and not very publishable problem space. The novelty in Kairos comes specifically from tying the two layers together — governed, live ML predictions and explanations as the substrate for a hybrid conversational retrieval system — and should be framed as such in your paper. Note that this novelty case matters most for the publication and for interview differentiation — it lives in Module B; your core Data Scientist story (Section 4.1) does not depend on it. Suggested contribution statements:

### 6.1 Explanation-aware retrieval

Most table-RAG systems retrieve rows and columns. Kairos additionally indexes and retrieves model-generated SHAP explanations and drift/model-confidence metadata as first-class, embeddable objects, so a conversational “why” question is answered by retrieving the actual explanation the model produced for that customer/segment — not by asking the LLM to reason freely (and potentially hallucinate) about feature importance.

### 6.2 Drift- and confidence-aware conversational answers

Kairos surfaces model staleness and confidence directly in the conversational answer (e.g., “this prediction is based on a model trained 41 days ago; feature drift has been detected in usage\_logs — treat this risk score with caution”), directly operationalising a risk that recent industry literature raises only conceptually.

### 6.3 A new evaluation axis: explanation faithfulness

Beyond standard text-to-SQL execution accuracy and RAGAS-style faithfulness/relevance, Kairos proposes and measures explanation faithfulness — whether a natural-language “why did this customer get flagged” answer correctly reflects the model's top-k SHAP drivers for that customer, scored automatically against the stored `churn_predictions.top_features` field.

## 6.4 One conversational surface, two audiences, one governed source of truth

Rather than building separate assistants per dashboard, Kairos unifies the operational (Power BI) and executive (Tableau) audiences behind a single chat interface reading from the same governed Postgres store — a concrete instance of the “decision intelligence” convergence trend identified in current industry analysis, implemented and evaluated rather than just described.

Recommended paper title to work toward: “Kairos: A Drift-Aware Conversational Analytics Layer for Explainable Customer Churn Intelligence via Hybrid Retrieval-Augmented Generation” (or your formal project title, Section 0). Position it as a systems/applied-AI paper (architecture + evaluation), not a pure NLP paper — this fits your timeline and skillset better than trying to beat text-to-SQL benchmark leaderboards.

## 7. System Architecture

---

### 7.1 Layered view

- Data & Ingestion Layer: Telco Customer Churn seed dataset (Kaggle) plus synthetically generated `usage_logs`, `support_tickets` and `transactions` (behaviour conditioned on churn outcome, not random), loaded via a Python/pandas ETL into PostgreSQL.
- Governed Storage Layer: PostgreSQL schema — `customers`, `subscriptions`, `usage_logs`, `support_tickets`, `transactions`, `churn_predictions` (with a `top_features` JSONB SHAP column), `model_registry` — plus pgvector extensions for embedded free-text columns and embedded SHAP explanation text.
- ML Layer: feature engineering with leakage discipline (fixed snapshot cutoff per customer, time-based train/test split), baseline Logistic Regression + XGBoost/LightGBM, Optuna/GridSearch tuning, SHAP global + local explanations, MLflow experiment tracking, PSI/KS-test drift monitoring, champion/challenger promotion.
- Conversational Retrieval Layer: automatic per-column profiling and routing — numeric/categorical/datetime columns queried via generated SQL (SQLAlchemy-executed), free-text and SHAP-explanation columns embedded via sentence-transformers into pgvector; a query classifier routes each natural-language question to SQL, semantic, or hybrid retrieval.
- Answer Synthesis Layer: an LLM composes the final grounded answer from retrieved SQL results, retrieved semantic context, and retrieved explanation/drift metadata, always returning the underlying SQL/context alongside the answer for auditability.

- **Serving Layer:** Flask + SQLAlchemy REST API (Flask-RESTX/Smorest for OpenAPI docs, Flask-JWT-Extended for auth, Pytest for coverage) exposing predict, batch-predict, customer-detail, segment, model-metrics, and chat/query endpoints.
- **Presentation Layer:** Power BI (operations dashboard: at-risk list, agent workload, drill-through) and Tableau (executive dashboard: churn trend, cohort retention, revenue-at-risk, aggregated SHAP drivers), plus a lightweight chat/web frontend that sends questions to the chat/query endpoint and can be embedded next to either dashboard.
- **Evaluation Layer:** a benchmark harness tracking text-to-SQL execution accuracy, RAGAS-style faithfulness/answer-relevance, and the new explanation-faithfulness metric, logged over time like the model metrics.

## 7.2 Representative data flow

A support manager asks, in the chat panel next to the Power BI dashboard, “Why is customer 4471 flagged high risk, and what should I do?” — the query classifier detects this as a hybrid + explanation query; the router pulls the customer's churn\_predictions row (SQL) and its stored SHAP top\_features (retrieved as an embedded explanation object), checks model\_registry for staleness/drift flags, and the LLM composes an answer citing the actual top-3 drivers, the model's current drift status, and — optionally — a recommended retention action mapped from the segment table, with the underlying SQL query and retrieved context shown for auditability.

## 8. Technology Stack

| Layer           | Technologies                                                                                                                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Database        | PostgreSQL + pgvector extension                                                                                                                                 |
| ML / XAI        | scikit-learn, XGBoost/LightGBM, SHAP, Optuna, MLflow, SciPy (PSI/KS-test)                                                                                       |
| RAG / LLM       | sentence-transformers (embeddings), LangChain or a lightweight custom orchestrator, provider-agnostic LLM layer (.env-configurable: OpenAI API or local Ollama) |
| Backend API     | Flask, SQLAlchemy, Flask-RESTX/Flask-Smorest, Flask-JWT-Extended, Pytest                                                                                        |
| BI / Dashboards | Power BI (operations), Tableau (executive)                                                                                                                      |
| DevOps          | Docker & docker-compose, GitHub Actions (CI for tests + eval harness)                                                                                           |
| Evaluation      | Custom text-to-SQL execution-accuracy harness, RAGAS, custom explanation-faithfulness scorer                                                                    |

## 9. Phased Project Timeline

The plan below is split into two execution blocks matching your academic calendar: Semester VII (August–October 2026), ending with your literature-review-plus-prototype checkpoint in early October, and Semester VIII (January–March 2027), ending with full submission at the end of March. Ten phases (0–9) in total — enough to give each skill area (data engineering, ML/XAI, conversational retrieval, MLOps, evaluation, writing) its own dedicated block, without fragmenting the work so much that no phase feels complete. Each phase lists what you build and what you must have ready to show.

## 9.1 Semester VII — August 2026 to early October 2026 ( $\approx$ 9 weeks)

| Phase                                                  | Timeframe                           | Activities & Deliverables                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 0<br>Kickoff                                     | Week 1 (early Aug)                  | Finalize the project charter with your guide, lock the Kairos problem statement and two-layer scope, set up GitHub repo and project board, and adopt this documentation as the living reference.                                                                                                                                                                                                                                                                                                                                                  |
| Phase 1<br>Literature<br>Review &<br>Need Study        | Weeks 1–3 (Aug)                     | Collect and read 25–35 papers across the three themes in Section 5 (start from the verified list in Section 15); write the formal literature review chapter; run a small market/need study (analyse 20–30 job postings for BI/ML/LLM-analytics roles + a short survey of 15–25 target users — support agents/managers — on dashboard pain points); finalize the novelty statement and draft paper title/abstract.                                                                                                                                 |
| Phase 2<br>Requirements<br>& Design                    | Weeks 3–4 (Aug–<br>Sep)             | Write the SRS; finalize ER diagram and full architecture diagram (Section 7); define API contracts; lock the tech stack and evaluation metrics; get guide sign-off on scope for Sem VII prototype.                                                                                                                                                                                                                                                                                                                                                |
| Phase 3 Core<br>Build —<br>Model, SHAP<br>& Dashboard  | Weeks 4–7 (Sep)                     | Stand up PostgreSQL schema; generate the synthetic usage/support/transaction data conditioned on churn outcome; build feature engineering with leakage checks; train baseline Logistic Regression + XGBoost with SHAP and derive the first driver/cohort/segment insights (Section 4.1); build one working dashboard (Power BI, ops view) wired to real predictions; skeleton Flask API (predict, customer-detail). Module B (bonus, if time allows): column auto-router + basic structured NL-to-SQL query over churn_predictions and customers. |
| Phase 4 Sem<br>VII Checkpoint<br>— Package &<br>Submit | Weeks 7–9 (late Sep<br>– early Oct) | Freeze the working prototype (see viva checklist below); write and finalize the interim report; draft a short/work-in-progress conference paper from the literature review + system design + Phase 3 results and submit it to a Scopus-indexed or IEEE conference (Track 1, Section 11) so that “paper submitted, under review” is a real, truthful line in your viva; rehearse the presentation with an explicit “what's done vs. what's coming in Sem VIII” roadmap slide.                                                                      |

### Sem VII Viva Checklist — What “Done” Looks Like

Even though Kairos is a two-semester project, the checkpoint should feel complete on its own terms — a smaller, working version of the final product, not a pile of unfinished pieces. Aim to walk into the viva with all of the following, truthfully:

- Research done: a finished literature review chapter (25+ papers, synthesized by theme, ending in a stated research gap) and a completed market/need study (job-posting analysis + small user survey) — this directly answers “have you done the research?”
- Planning done: SRS, ER diagram, and full architecture diagram, all guide-approved, showing the complete Sem VII + Sem VIII scope so the panel sees this checkpoint as a milestone inside a bigger plan, not the whole plan.
- Working product (small but real): a trained churn model with SHAP explanations, at least 3–4 concrete business insights written up in the Section-4.1 style, and one live dashboard (Power BI ops view) showing real predictions — something you can click through in front of the panel, not just slides.
- Bonus, if it lands in time: a basic structured natural-language query demo (“show me all high-risk customers in the West region” → generated SQL → real answer) as a preview of Module B. If it isn't ready, say so plainly and show it as a Sem VIII deliverable instead — do not fake or rush this for the viva.
- Publication status: a short/WIP paper submitted (or, at minimum, a complete, ready-to-submit manuscript with a named target venue) — framed explicitly as “Sem VIII will extend this with full results and the hybrid RAG module for a full paper submission.”
- A one-slide roadmap: a simple “what you're seeing today vs. what's coming in Sem VIII” slide (drift monitoring, second dashboard, hybrid RAG + explanation-faithfulness evaluation, full paper) — this reassures the panel that the incompleteness is planned, not accidental.

## 9.2 Semester VIII — January 2027 to end of March 2027 (≈ 12 weeks)

| Phase                                            | Timeframe            | Activities & Deliverables                                                                                                                                                                                                                                                     |
|--------------------------------------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 5 Hybrid Retrieval + Explanation-Aware RAG | Weeks 1–3 (Jan)      | Add pgvector embeddings for free-text support-ticket content and for stored SHAP explanations; implement the full query router (aggregation / lookup / semantic / hybrid); wire SHAP + drift metadata into the retrieval context so “why” questions are explanation-grounded. |
| Phase 6 Drift, MLOps & BI Integration            | Weeks 3–6 (Jan–Feb)  | Add MLflow tracking, PSI/KS drift monitoring, champion/challenger promotion; build Power BI (ops) and Tableau (exec) dashboards reading from the same governed Postgres store; embed the chat interface alongside both dashboards.                                            |
| Phase 7 Evaluation Harness                       | Weeks 6–8 (Feb)      | Build the curated question/SQL evaluation set; measure text-to-SQL execution accuracy, RAGAS-style faithfulness/relevance, and the new explanation-faithfulness metric; iterate on the router/prompting based on results; add JWT auth, tests, and Docker containerization.   |
| Phase 8 Paper Drafting & Submission              | Weeks 8–10 (Feb–Mar) | Draft the full paper (Intro, Related Work from Section 5, System, Evaluation, Results, Conclusion); select target venue(s) per Section 11; submit.                                                                                                                            |

|                                   |                        |                                                                                                                                           |
|-----------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 9 Final Report & Submission | Weeks 10–12 (Mar, end) | Finalize the full project report, deployment, and demo video/poster; final presentation and viva; submit everything by end of March 2027. |
|-----------------------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|

Note on the January gap: nothing needs to happen on Kairos itself between early October and January — just keep a running notes file of any new papers, job postings, or ideas you come across, so Phase 5 doesn't start cold.

## 10. Recommended Learning Resources (YouTube) — by Phase

---

These are starting points to build each piece of Kairos, mapped to the phase where you'll actually need them. Skim the first few minutes of each before committing to the full video to make sure the pace and depth fit you — these are general tool/technique tutorials, not Kairos-specific, so you will still need to adapt the code to your own schema and architecture.

### Phase 3 — Core Build: Model, SHAP & Dashboard

- Understanding Model Predictions with SHAP — XGBoost vs Neural Networks:  
<https://www.youtube.com/watch?v=eLrtufRomh8> — walks through SHAP for tree-based models, directly applicable to your churn model.
- Power BI Dashboard for Customer Churn Analysis | Insights from Machine Learning:  
<https://www.youtube.com/watch?v=LXRhI3oUno> — closest match to Kairos, since it builds the dashboard from model predictions rather than raw data.
- Customer Churn Dashboard End to End Tutorial in Power BI:  
<https://www.youtube.com/watch?v=ImBgUbRrw3A> — good for KPI-card and interactive-filter mechanics for the ops dashboard.
- Creating a RESTful API in Flask with JSON Web Token Authentication and Flask-SQLAlchemy:  
<https://www.youtube.com/watch?v=WxGBoY5iNXY> — for the skeleton Flask API (predict, customer-detail endpoints).

### Phase 5 — Hybrid Retrieval + Explanation-Aware RAG

- PGVECTOR — Vector Search in PostgreSQL Using pgvector: A Complete Tutorial:  
<https://www.youtube.com/watch?v=wsBGpWBo0Ks> — covers embeddings and similarity search directly in Postgres, matching Kairos's architecture (no separate vector database).

### Phase 6 — Drift, MLOps & BI Integration

- MLflow Python Tutorial — ML Model Experiment Tracking:  
<https://www.youtube.com/watch?v=kn51WgTTjCw>
- Track Your ML Experiments with MLflow | Quick Tutorial:  
<https://www.youtube.com/watch?v=Z4unUK0vn4k> — use alongside the above for the model registry and champion/challenger workflow.

- For the Tableau executive dashboard specifically, search current Tableau tutorials directly — the Power BI videos above are useful for dashboard-design mechanics (KPI cards, drill-through, filters), but the tooling differs enough that a Tableau-specific walkthrough is worth finding closer to when you build it.

### Phase 7 — Evaluation Harness

- How to Evaluate RAG Systems | RAGAS Framework Explained | ML Interview: [https://www.youtube.com/watch?v=ODEC\\_6lRetM](https://www.youtube.com/watch?v=ODEC_6lRetM) — recent (Jan 2026), covers RAGAS metrics you'll use for faithfulness/relevance scoring.
- Evaluating RAG Applications in Minutes Using RAGAs!: <https://www.youtube.com/watch?v=-69Fx8F9ma4>
- JWT Authentication for Flask with Flask-JWT-Extended (2-hour crash course): [https://www.youtube.com/watch?v=aX-ayOb\\_Aho](https://www.youtube.com/watch?v=aX-ayOb_Aho) — for hardening auth and adding tests before containerization.

## 11. Publication Plan — Targeting Scopus / IEEE

---

An honest constraint worth planning around: traditional Scopus-indexed journal peer review commonly takes anywhere from 3 to 9+ months, which does not reliably fit inside your March 2027 deadline if you only start submitting in February. Scopus-indexed and IEEE conference proceedings typically review much faster (4–10 weeks) and are the more realistic route to have an actual indexed publication in hand for job applications by the time you graduate. A sensible dual-track plan:

1. Track 1 — Conference paper (primary target, moved earlier so it's real by your viva): as soon as the literature review is finished and Phase 3 gives you a trained model with SHAP results and one working dashboard, write a shorter paper (system design + preliminary results) and aim to submit it to a Scopus-indexed or IEEE conference by the end of Phase 4, i.e. by your Sem VII checkpoint in late September/early October 2026. This means you can honestly tell the viva panel the paper is submitted and under review, not just planned. Look for AI/data-science/software-engineering conferences with Springer or IEEE proceedings; search current calls on Springer's conference portal and IEEE Conferences, since call-for-papers deadlines change often — check these close to submission time rather than relying on this document, and pick a venue whose deadline actually falls before your checkpoint.
2. Track 2 — Full journal paper (stretch target): once the evaluation harness (Phase 7) produces real numbers, write the full paper with complete results and submit to a Scopus-indexed journal around February–March 2027. Even if the decision arrives after graduation, “under review” / “communicated” is a legitimate and common resume line for new graduates.
3. Candidate venue families to shortlist: for journals — International Journal of Data Science and Analytics (Springer), Journal of Big Data (Springer, open access), and Results in Engineering (Elsevier), since papers you'll cite in Section 15 already appear there; for IEEE, watch for regional/national conferences on AI, data science, or software engineering that run through IEEE

Xplore (many list churn prediction and RAG-adjacent tracks). Verify current Scopus status and APC/waiver policy directly on each publisher's site before submitting, since indexing and fees change.

4. Whichever venue you choose, verify its live Scopus status yourself on [scopus.com](https://scopus.com) or the Scimago Journal Rank site, and its IEEE Xplore status on [ieeexplore.ieee.org](https://ieeexplore.ieee.org), just before submission — indexing status and APC policy can change, and this document may be stale by then.

## 12. Evaluation Methodology

---

- Model quality: ROC-AUC, PR-AUC (churn is imbalanced), calibration check, plus a business metric — expected revenue saved by acting on the top-risk decile.
- Text-to-SQL quality: execution accuracy against a curated question/gold-SQL test set (extend with deliberately ambiguous/unanswerable questions, following the PRACTIQ dataset design).
- RAG answer quality: RAGAS-style faithfulness and answer-relevance scoring for semantic and hybrid questions.
- Explanation faithfulness (novel metric): automatically compare the top drivers mentioned in a conversational “why” answer against the ground-truth SHAP top\_features stored for that prediction, reporting overlap/precision.
- Drift-awareness check: verify the system correctly surfaces a staleness/drift warning when model\_registry / drift-monitoring flags are active, and does not overstate confidence when they are.
- User study (optional, strengthens the paper): a short task-based usability comparison — time-to-insight and correctness — for a handful of ops/exec users using the chat interface vs. the raw dashboards.

## 13. Expected Outcomes & Deliverables

---

- A governed, production-style relational churn database (not a single flat CSV).
- An interpretable churn model with per-customer SHAP explanations, MLflow tracking, and drift monitoring.
- A hybrid SQL + semantic conversational query engine that is explanation- and drift-aware.
- Two audience-specific BI dashboards (Power BI, Tableau) unified behind one conversational surface.
- A JWT-secured, tested, containerized Flask REST API.
- A reproducible evaluation report (text-to-SQL accuracy, RAGAS metrics, explanation-faithfulness).
- A completed literature review, market/need study, full project report, and a submitted (ideally accepted) Scopus-indexed or IEEE paper.
- A portfolio- and interview-ready capstone with a clear, defensible novelty story for placements.

## 14. Draft Resume Bullets

---

Two versions — lead with whichever matches the role. For classic Data Scientist applications, use the first as your primary bullet and the second as a supporting line, not the reverse.

**Kairos — Explainable Customer Churn Intelligence Platform:** Built and validated an end-to-end churn prediction system (XGBoost, SHAP-explainable, MLflow-tracked, drift-monitored) on a governed PostgreSQL schema; translated model outputs into segment-level retention recommendations and revenue-at-risk estimates, delivered through operations (Power BI) and executive (Tableau) dashboards for two distinct stakeholder audiences.

**Kairos — Conversational Analytics Extension:** Additionally layered a hybrid text-to-SQL + pgvector retrieval-augmented conversational interface on top of the platform so non-technical stakeholders could query churn data and model explanations in plain English; evaluated with a custom benchmark combining text-to-SQL execution accuracy, RAGAS-style faithfulness, and a novel explanation-faithfulness metric; results submitted to a Scopus-indexed or IEEE venue.

## 15. Reference List — Verified IEEE Xplore / Scopus-Indexed Sources

---

These are real, verifiable IEEE Xplore documents and Scopus-indexed journal articles, organized by the three literature-review themes in Section 5. IEEE Xplore document numbers and DOIs are given where available — look these up directly on [ieeexplore.ieee.org](https://ieeexplore.ieee.org) or via the DOI to pull full bibliographic details (volume, pages, exact publication date) for your citation style. Verify each entry's live indexing status before your final submission, since coverage can change.

### 15.1 Theme A — Explainable Churn Prediction

1. Asif, D., Arif, M.S., Mukheimer, A. (2025). A data-driven approach with explainable artificial intelligence for customer churn prediction in the telecommunications industry. Results in Engineering, 26, 104629. Elsevier (Scopus-indexed). DOI: 10.1016/j.rineng.2025.104629.
2. El Attar, A., El-Hajj, M. (2026). Explainable AI-driven customer churn prediction: a multi-model ensemble approach with SHAP-based feature analysis. Frontiers in Artificial Intelligence. DOI: 10.3389/frai.2026.1748799. (Scopus / PubMed indexed)
3. Bogaert, M., Delaere, L. (2023). Ensemble methods in customer churn prediction: a comparative analysis of the state-of-the-art. Mathematics, 11(5), 1137. MDPI (Scopus-indexed).
4. Chang, V., Hall, K., Xu, Q.A., Amao, F.O., Ganatra, M.A., Benson, V. (2024). Prediction of customer churn behavior in the telecommunication industry using machine learning models. Algorithms, 17(6), 231. MDPI (Scopus-indexed). DOI: 10.3390/a17060231.
5. Christopher, D., Anand, G. (2024). Comparative analysis of predictive models for customer churn prediction in the telecommunication industry. 2024 International Conference on Communication,

Computer Sciences and Engineering (IC3SE), IEEE, 534–539. DOI: 10.1109/IC3SE62002.2024.10592931. (IEEE Xplore)

6. Customer Churn Prediction Using Machine Learning and Explainable AI. IEEE Conference Publication, IEEE Xplore, Document 11412808.
7. Explainable Machine Learning: A Manuscript on the Customer Churn in the Telecommunications Industry. IEEE Conference Publication, IEEE Xplore, Document 9708561.
8. Churn Prediction With Explainability for the Customers of Telecom Industry. IEEE Conference Publication, IEEE Xplore, Document 10303395.
9. An Improved Machine Learning Based Customer Churn Prediction for Insight and Recommendation in E-commerce. IEEE Conference Publication, IEEE Xplore, Document 10054771.
10. Explainable Artificial Intelligence for Data Science on Customer Churn. IEEE Conference Publication, IEEE Xplore, Document 9564166.

## 15.2 Theme B — Natural-Language-to-SQL and Conversational Database Interfaces

1. Querylizer: An Interactive Platform for Database Design and Text to SQL Conversion. IEEE Conference Publication, IEEE Xplore, Document 9725828.
2. Conversion of Natural Language Query to SQL Query. IEEE Conference Publication, IEEE Xplore, Document 8474639.
3. SPSQL: Step-by-step Parsing Based Framework for Text-to-SQL Generation. IEEE Conference Publication, IEEE Xplore, Document 10148562.
4. Synthesis of SQL Queries from Narrations. IEEE Conference Publication, IEEE Xplore, Document 9004293.
5. Chat2DB: Chatting to the Database with Interactive Agent. 2025 IEEE 41st International Conference on Data Engineering (ICDE). IEEE Xplore, Document 11113201.

## 15.3 Theme C — Retrieval-Augmented Generation over Structured/Enterprise Data

1. Structured Retrieval-Augmented Generation for Multi-Entity Question Answering over Heterogeneous Sources. IEEE Conference Publication, IEEE Xplore, Document 11107459.
2. Agentic Retrieval-Augmented Generation for On-Premise Infrastructure: Enhancing Security and Performance for Enterprise Knowledge Management. IEEE Conference Publication, IEEE Xplore, Document 11436668.
3. Metadata-based Data Exploration with Retrieval-Augmented Generation for Large Language Models. IEEE Conference Publication, IEEE Xplore, Document 10826055.

## 15.4 Supplementary / Context Reading — Not Confirmed Scopus/IEEE-Indexed

Useful for conceptual grounding and for building your “research gap” narrative, but these are arXiv preprints, industry reports, or ACL Anthology proceedings — verify indexing status before citing them formally in a Scopus/IEEE submission:

1. Gao, Y., Xiong, Y., et al. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.
2. Sharma, C. (2025). Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers. arXiv:2506.00054.
3. Nahid, M.M.H., Rafiei, D., Zhang, W., Zhang, Y. (2025). Rethinking Schema Linking: A Context-Aware Bidirectional Retrieval Approach for Text-to-SQL. arXiv:2510.14296.
4. Dong, M., Ashok Kumar, N., Hu, Y., et al. (2025). PRACTIQ: A Practical Conversational Text-to-SQL Dataset with Ambiguous and Unanswerable Queries. Proc. NAACL 2025 (Vol. 1: Long Papers), 255–273. (ACL Anthology — verify Scopus coverage for this venue/year before citing as Scopus-indexed.)
5. Conversational Natural Language Interfaces Are the New Front End for Governed Predictive Analytics (2026). Industry analysis — not a peer-reviewed academic source; use for context only.
6. Natural Language Analytics: From SQL to Conversation (2026). Industry report on decision intelligence and conversational analytics adoption — not a peer-reviewed academic source; use for context only.

## 16. How to Use This Document

---

Treat this as the living master reference for Kairos — share it with your guide as-is for scope approval, then split it into your institute's required formats (SRS, interim report, final report) as each phase completes. Sections 5, 6, 10, and 15 are the parts to revisit most often, since your literature review, novelty framing, learning needs, and target venue will sharpen as you read more and get results.